ECE551: Digital System Design and Synthesis

Fall 2026

Testbench

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Generating Clocks

- Wrong way…

```
initial begin
  #5 clk = 0;
  #5 clk = 1;
  #5 clk = 0;
  … (repeat hundreds of times)
end
```

- Right way

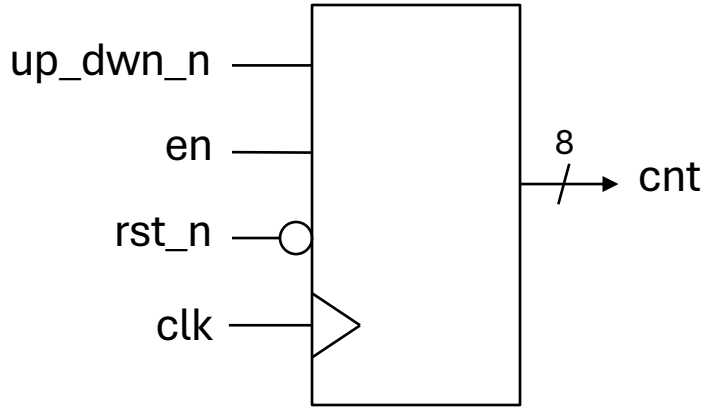
```
initial clk = 0;

always
  #5 clk = ~clk;
```

```
initial begin
  clk = 0;
  forever #5 clk = ~clk;
end
```

Stimulus for Sequential Circuits

- Imagine an 8-bit counter with an active low reset
- When en=0, the count is frozen
- The counter counts up if up_dwn_n=1, counts down if up_dwn_n=0



- What should the testbench look like?
- How should the stimulus be generated?

Stimulus for Sequential Circuits

```
module up_dwn_tb();

logic clk, rst_n;
logic en, up_dwn_n;

// Instantiate DUT //
up_dwn iDUT(.clk(clk), .rst_n(rst_n), .en(en), .up_dw

initial begin
  clk = 0;
  rst_n = 0;          // assert reset
  en = 0;             // start with it disabled
  up_dwn_n = 1;       // count up at first
  #16 rst_n = 1;      // deassert clock
  #5 en = 1;          // start counting
  #50 en = 0;         // stop after 5 clocks
  #10 en = 1;         // start after 1 clock freeze
  #20 up_dwn_n = 0; // start counting down
end

always
  #5 clk = ~clk;

endmodule
```

- Can do it using delays shown here
- Signal timings are calculated based on your clock period
- Figuring it out is a pain
- What if you want to change the clock period?

- There is a better way…

Stimulus for Sequential Circuits

```
module up_dwn_tb();

logic clk, rst_n;
logic en, up_dwn_n;

// Instantiate DUT //
up_dwn iDUT(.clk(clk), .rst_n(rst_n), .en(en), .up_dwn_n

initial begin
  clk = 0;
  rst_n = 0;                    // assert reset
  en = 0;                       // start with it disabled
  up_dwn_n = 1;                 // count up at first
  @(posedge clk);               // wait one clock cycle
  @(negedge clk) rst_n = 1;     // deassert reset on negativ
  @(posedge clk) en= 1;         // start counting
  repeat (5)@(posedge clk);     // wait 5 clock cycles
  en = 0;                       // stop counting
  @(posedge clk) en = 1;        // for 1 clock cycle
  repeat(2) @(posedge clk);     // wait 2 clock cycles
  up_dwn_n = 0;                 // start counting backwards
end

always
  #5 clk = ~clk;
```

- Can use clock edges as trigger for stimulus events
- Easier to think about, no arithmetic with clock period
- Code is independent of clock period
- Can use repeat() loop to wait a given number of clocks

Exhaustive Testing

- Practical for combinational designs w/ up to 8 or 9 inputs
  - Test ALL combinations of inputs to verify output
  - Could enumerate all test vectors, but don't…
  - Generate them using a for loop!

```
logic [4:0] x;
initial begin
  for (x = 0; x < 16; x = x + 1)
    #5; // need a delay here!
end
```

Loop vector needs an extra bit

- Want to test all vectors 0000 to 1111
- If x is 4 bits, you can only go up to 1111
  - X is never >= 16
  - Loop goes forever

Force/Release

- Allows you to “override” value **for simulation**
- How does this help test?
  - Can create a “short cut” to a state that is difficult to get to through normal paths
  - Examples:
    - Force a state in a SM that would take many sequences to get to through “normal operation”
    - Force an error condition in a protocol if you want to test how your error correction mechanism worked
  - Can help achieve code coverage

Force/Release Example

- Toy example showing force/release mechanism

```
assign y = a & b;
assign z = y | c;
initial begin
    a = 0; b = 0; c = 0;
    #5 a = 0; b = 1; c = 0;
    #5 force y = 1;
    #5 b = 0;
    #5 release y;
    #5 $stop;
end
```

T	a	b	c	y	z
0	0	0	0	0	0
5	0	1	0	0	0
10	0	1	0	1	1
15	0	0	0	1	1
20	0	0	0	0	0